

Algorithms and Data Structures II.

Lecture Notes: Trees

Tibor Ásványi

Department of Computer Science
Eötvös Loránd University, Budapest
asvanyi@inf.elte.hu

November 15, 2024

Contents

1	Introduction	4
2	AVL Trees	4
2.1	AVL trees: insertion	8
2.2	AVL trees: removing the minimal (maximal) node	17
2.3	AVL trees: deletion	20
3	General trees	23
4	B+ trees and their basic operations	25

References

- [1] ÁSVÁNYI, T, Algorithms and Data Structures I. Lecture Notes
<http://aszt.inf.elte.hu/~asvanyi/ds/AlgDs1/AlgDs1LectureNotes.pdf>
- [2] BURCH, CARL, B+ trees
(See <http://aszt.inf.elte.hu/~asvanyi/ds/AlgDs2/B+trees.pdf>)
- [3] CORMEN, T.H., LEISERSON, C.E., RIVEST, R.L., STEIN, C.,
Introduction to Algorithms (Third Edition), *The MIT Press*, 2009.
- [4] CORMEN, THOMAS H., Algorithms Unlocked, *The MIT Press*, 2013.
- [5] NARASHIMA KARUMANCHI,
Data Structures and Algorithms Made Easy, *CareerMonk Publication*,
2016.
- [6] NEAPOLITAN, RICHARD E., Foundations of algorithms (Fifth edition),
Jones & Bartlett Learning, 2015. ISBN 978-1-284-04919-0 (pbk.)
- [7] SHAFFER, CLIFFORD A.,
A Practical Introduction to Data Structures and Algorithm Analysis,
Edition 3.1 (C++ Version), 2011
(See <http://aszt.inf.elte.hu/~asvanyi/ds/C++3e20110103.pdf>)
- [8] WEISS, MARK ALLEN, Data Structures and Algorithm Analysis in C++
(Fourth Edition),
Pearson, 2014.
- [9] WIRTH, N., Algorithms and Data Structures,
Prentice-Hall Inc., 1976, 1985, 2004.
(See <http://aszt.inf.elte.hu/~asvanyi/ds/AD.pdf>)

1 Introduction

We need *dictionaries*, i.e. large sets of data with efficient insertion, search and deletion operations (where removing the maximal or minimal element of the dictionary and removing a piece of data with a given key are the cases of deletion). We already have two solutions where the average performance (AT) of these operations is good, but the worst-case performance (MT) is too slow (n is the size of the set/data structure):

- Binary search trees (BSTs): $AT(n) \in \Theta(\log n)$, $MT(n) \in \Theta(n)$.
- Hash tables: $AT(n) \in \Theta(1)$ (if the load factor is not too high), $MT(n) \in \Theta(n)$. (Removing the maximal and minimal elements is not supported.)

We are going to discuss some kinds of *balanced search trees* where we can guarantee $MT(n) \in \Theta(\log n)$ for each dictionary operation.

- *AVL trees*: balanced BSTs for storing data in the central memory.¹
- *B+ trees*: perfectly balanced multiway search trees optimised for storing data on hard disks.²

In the next section, we consider *AVL trees*. In the last one, we describe *B+ trees*.

2 AVL Trees

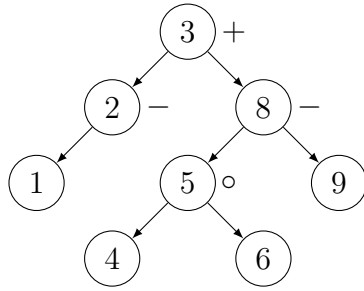
Being BSTs, AVL trees have linked representation typically. In these notes, we suppose that they have linked representation. Everywhere letter n denotes the tree's size (number of nodes), and h represents the tree's height.

Definition 2.1 *The balance of node $(*p)$ of a binary tree is $p \rightarrow b = h(p \rightarrow \text{right}) - h(p \rightarrow \text{left})$. This means that the balance of a node is the height of its right subtree minus the height of its left subtree.*³

¹*Red-black trees* are also specially balanced BSTs and can provide another efficient solution.

²*B trees* are an older solution.

³The *balance* notion above is sometimes called *height-balance*, in order to distinguish it from other kinds of *balances*. For example, a node's *size-balance* is the size of its right subtree *minus* the size of its left subtree. A node is *size-balanced*, **iff** its *size-balance* is in $\{-1, 0, 1\}$. A tree is *size-balanced*, **iff** all of its nodes are *size-balanced*. In these notes, *balance* means *height-balance* by default.



The binary tree on the left in

- simple textual form:
 $(((1) 2- ()) 3+ (((4) 5o (6)) 8- (9)))$
- elegant parenthesised form:
 $\{ [(1) 2- ()] 3+ [((4) 5o (6)) 8- (9)] \}$

Figure 1: The same binary tree in graphical and textual representations. The balance of each leaf is zero, so we omit the balances of the leaves. We show the balance of each internal node next to it (or next to the key identifying it). In the example trees, we use the following notation for the balances: $0 \sim o$; $1 \sim +$; $2 \sim ++$; $-1 \sim -$; $-2 \sim --$

Example 2.2 See the BST on Figure 1. Deleting node ① from it, we receive BST $\{ [2] 3++ [((4) 5o (6)) 8- (9)] \}$. And inserting node ⑦ into the BST on Figure 1, we receive the following BST.
 $\{ [(1) 2- ()] 3++ [((4) 5+ \langle \{ \} \rangle 6+ \{7\}) 8-- (9)] \}$.

Exercise 2.3 Draw each tree, which is given in textual representation in these lecture notes. Drawing them provides easier and deeper understanding and better long-term knowledge. One can see a pattern above, at Figure 1.

Definition 2.4 Node $(*p)$ is perfectly balanced, **iff** its balance, $p \rightarrow b = 0$.⁴

Definition 2.5 Node $(*p)$ is balanced, **iff** its balance, i.e. $p \rightarrow b \in \{-1, 0, 1\}$. Otherwise, it is imbalanced.

Consequence 2.6 The leaves of a binary tree are perfectly balanced because both the right and left subtrees of a leaf are empty.

Definition 2.7 A binary tree is balanced, **iff** each (internal) node of it is balanced. It is imbalanced, **iff** at least one of its nodes is imbalanced.

Definition 2.8 A binary tree is perfectly balanced, **iff** each (internal) node of it is perfectly balanced.

Consequence 2.9 A binary tree is perfectly balanced, **iff** it is complete.

⁴iff means **if and only if**.

Definition 2.10 An AVL tree is a balanced BST.⁵

Remember that $MT(h) \in \Theta(h)$ is valid for the basic operations of BSTs (insertion, search and deletion). Fortunately, the BST and the balanced tree properties can be maintained efficiently throughout insertions and deletions on balanced BSTs, and the following theorem guarantees that the height of an AVL tree never goes far from the ideal case. Thus, we can ensure the high efficiency of these operations on AVL trees.

Theorem 2.11 Given a nonempty balanced binary tree,

$$\lfloor \log n \rfloor \leq h \leq 1.45 \log n, \quad \text{i.e.} \quad h \in \Theta(\log n)$$

We provide an outline of the proof of this theorem. The lower bound given there is the lower bound of the height of all the binary trees. Thus, it is also true for balanced binary trees. The upper bound shown here follows from the properties of *Fibonacci trees*.

Definition 2.12 A binary tree is Fibonacci tree, **iff** each internal node of it is balanced, but not perfectly balanced.

These are called *Fibonacci trees* because a Fibonacci tree with height $h > 0$ consists of a root *plus* a left subtree with height $h-1$ and a right subtree with height $h-2$ or vice versa where both subtrees are Fibonacci trees. And this recursive description is similar to the recursive definition of *Fibonacci numbers*.

It is easy to prove that the Fibonacci trees have the smallest size among the balanced binary trees with a given height. Let f_h be the size of a nonempty Fibonacci tree with height h . Clearly $f_0 = 1, f_1 = 2, f_h = 1 + f_{h-1} + f_{h-2} \quad (h \geq 2)$.

⁵It is easy to prove that the *size-balanced binary trees* are special cases of the *nearly complete binary trees*. And we know that the height h of a nearly complete binary tree is minimal among the binary trees of a given size n , i.e. $h = \lfloor \log n \rfloor$, plus $MT(h) \in \Theta(h)$ for the basic operations of BSTs (insertion, search and deletion). Consequently, these operations are most efficient on nearly complete, especially size-balanced BSTs. Thus, one may ask why we do not use *size-balanced* or *nearly complete* BSTs instead of AVL trees. Well, the *size-balanced* and the *nearly complete* properties are too strong requirements in most cases because we cannot maintain these properties efficiently throughout insertions and deletions on BSTs. Therefore, these properties are often lost throughout insertions and deletions, and the height of a resulting BST may go far from the ideal case. Thus, we use AVL trees because the AVL tree property can be maintained extremely efficiently throughout insertions and deletions, and the height of an AVL tree is quite close to the minimal height $h = \lfloor \log n \rfloor$ even in the worst case.

These formulas are similar to the definition of the Fibonacci sequence:

$$F_0 = 0, F_1 = 1, F_h = F_{h-1} + F_{h-2} \quad (h \geq 2).$$

It can be proved with mathematical induction that $f_h = F_{h+3} - 1$ where h is a natural number. $f_h \leq n$ where n is the size of a balanced binary tree with height h . We can get the upper bound given in the previous theorem with some mathematical skill. (We omit the details.)

From the previous results, it can be proved that $MT(n) \in \Theta(\log n)$ for the basic operations of AVL trees (insertion, search and deletion). We still have to see that $MT(h) \in \Theta(h)$ even for the AVL tree version of these operations where each operation keeps the AVL tree property.

To achieve this aim, first, we describe our representation of the nodes of AVL trees. Then, we develop the algorithms of the basic operations of the AVL trees. It will be apparent that these operations go down and possibly up only once in the tree, and the number of elementary transformations can be limited with the same constant at each level, so $MT(h) \in \Theta(h)$ is true. See the representation first.

Node
+ $key : \mathcal{T}$ // $\mathcal{T}$ is some known type
+ $b : -1..1$ // the balance of the node
+ $left, right : \text{Node}^*$
+ $\text{Node}() \{ left := right := \odot ; b := 0 \}$ // create a tree of a single node
+ $\text{Node}(x:\mathcal{T}) \{ left := right := \odot ; b := 0 ; key := x \}$

We can see that the balance of a node is stored explicitly in the node. If a tree is modified, the balances of some nodes change. Thus, we have to adjust these nodes' data members b . This process will be called *rebalancing* in these notes. When we illustrate an operation on a BST, we will show the balance values stored in the node objects. (Although a stored balance value and the corresponding proper one must be equal before and after the operation, they may differ during the transformation.)

Let us consider the efficiency and some details of the basic operations of AVL trees. Remember that the height of an AVL tree is $\Theta(\log n)$. Thus the functions $\text{search}(t, k)$, $\text{min}(t)$ and $\text{max}(t)$ of BSTs [1] (which do not modify the tree and go down in the tree once) can be applied to AVL trees with $MT(n) \in \Theta(\log n)$ efficiency.

Procedures $\text{insert}(t, k)$, $\text{del}(t, k)$, $\text{remMin}(t, \text{minp})$ and $\text{remMax}(t, \text{maxp})$ of BSTs [1] applied to AVL trees also run on AVL trees with $MT(n) \in \Theta(\log n)$ efficiency, and the result will be a BST, but possibly imbalanced. (See Example 2.2. The input of both operations is the same AVL tree, but

the outputs are imbalanced BSTs.) Clearly, after many modifications, the resulting tree may become too high for an efficient run of the operations of BSTs. Therefore, we need some changes on these recursive procedures: When we return from a recursive call, we check whether the actual node of the tree became imbalanced, and if so, we perform the appropriate *rotations* to make it balanced. The same constant at each level can limit the extra elementary operations needed, so efficiency $MT(n) \in \Theta(\log n)$ remains true.

The *rules of rotations* of AVL trees can be found in Figures 2-7. One can see that *each rotation* keeps the original inorder tree traversal. Consequently, if the input of a rotation is a BST, its output is also a BST.

After an addition or deletion, we consider the smallest imbalanced subtree containing the position of the insertion or deletion and apply the appropriate rotation to it. Thus, its subtrees are balanced, and the result of the rotation is a balanced subtree. Sometimes, additional rotations must be done at higher levels of the tree afterwards.

2.1 AVL trees: insertion

First, we consider procedure $\text{insert}(t, k)$ of BSTs [1].

For example, given AVL tree $\{ [2] \ 4+ \ [(6) \ 8 \circ (10)] \}$.

- Inserting key 1, we receive AVL tree $\{ [(1) \ 2- \ ()] \ 4 \circ [(6) \ 8 \circ (10)] \}$.
- Alternatively, inserting key 3, we receive AVL tree $\{ [() \ 2+ \ (3)] \ 4 \circ [(6) \ 8 \circ (10)] \}$.
- Alternatively, inserting key 9, we receive imbalanced BST $\{ [2] \ 4++ \ [(6) \ 8+ \ (\{9\} \ 10- \ \{ \})] \}$.
- Alternatively, inserting key 7, we receive imbalanced BST $\{ [2] \ 4++ \ [(\{ \} \ 6+ \ \{7\}) \ 8- \ (10)] \}$.

The tree became imbalanced in the last two cases. If we want to receive an AVL tree, we have to make the appropriate rotations on it. Let's see some detailed examples.

Example 2.13 *Insert key 3 into AVL tree $\{ [2] \ 4+ \ [(6) \ 8 \circ (10)] \}$.*

Solution: The key of the root of $\{ [2] \ 4+ \ [(6) \ 8 \circ (10)] \}$ is 4. $3 < 4$, we go to the left subtree: $[2]$.

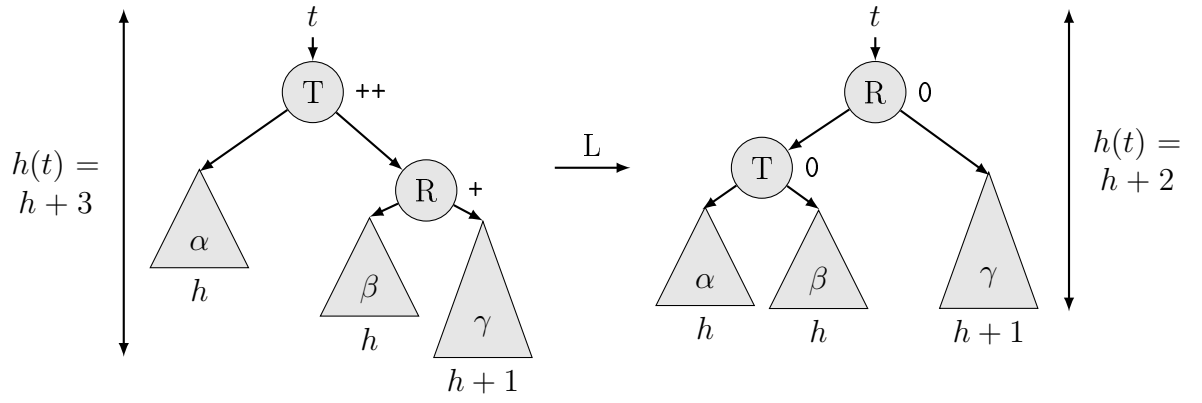


Figure 2: rotation $(++, +)$.

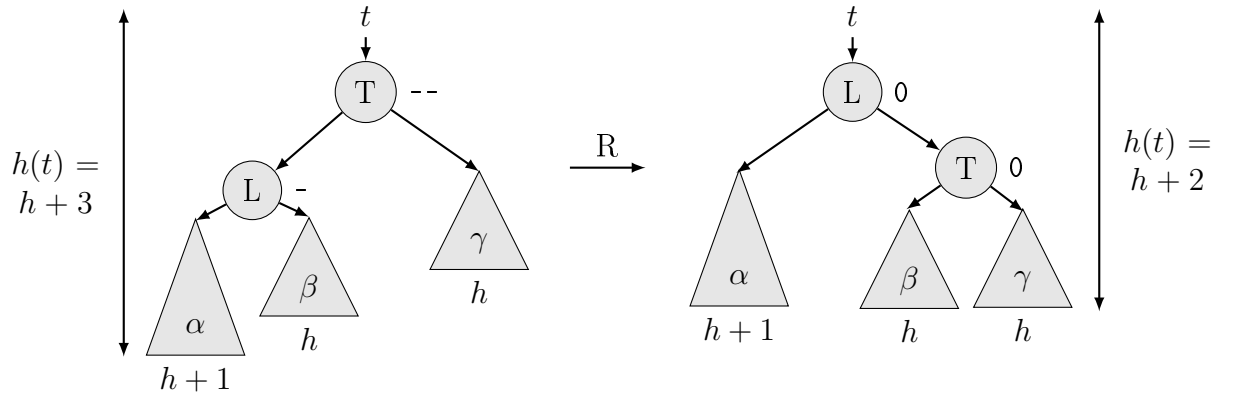


Figure 3: rotation $(--, -)$.

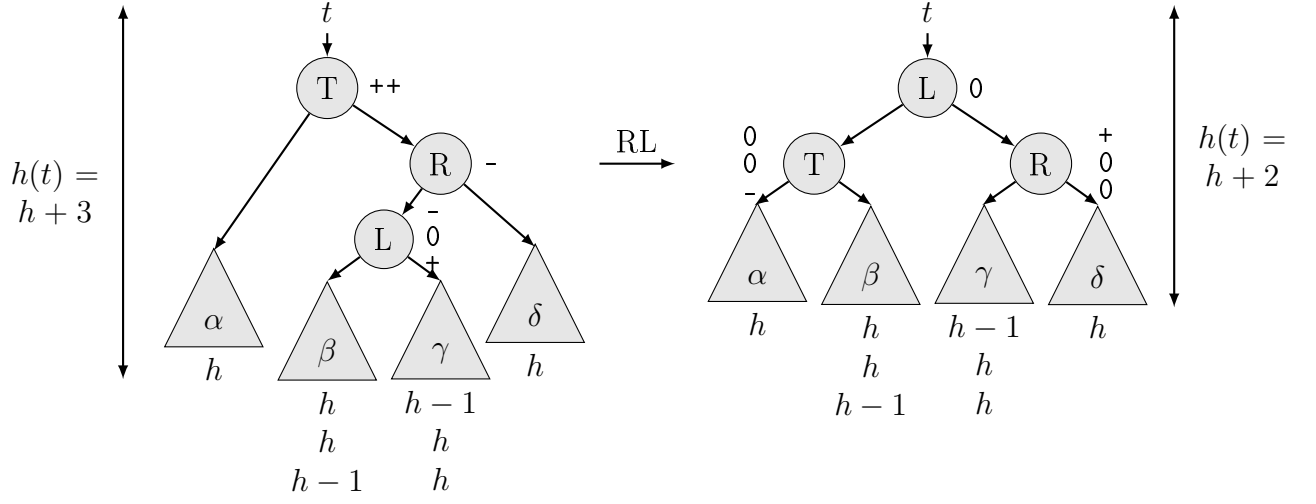


Figure 4: rotation $(++, -)$.

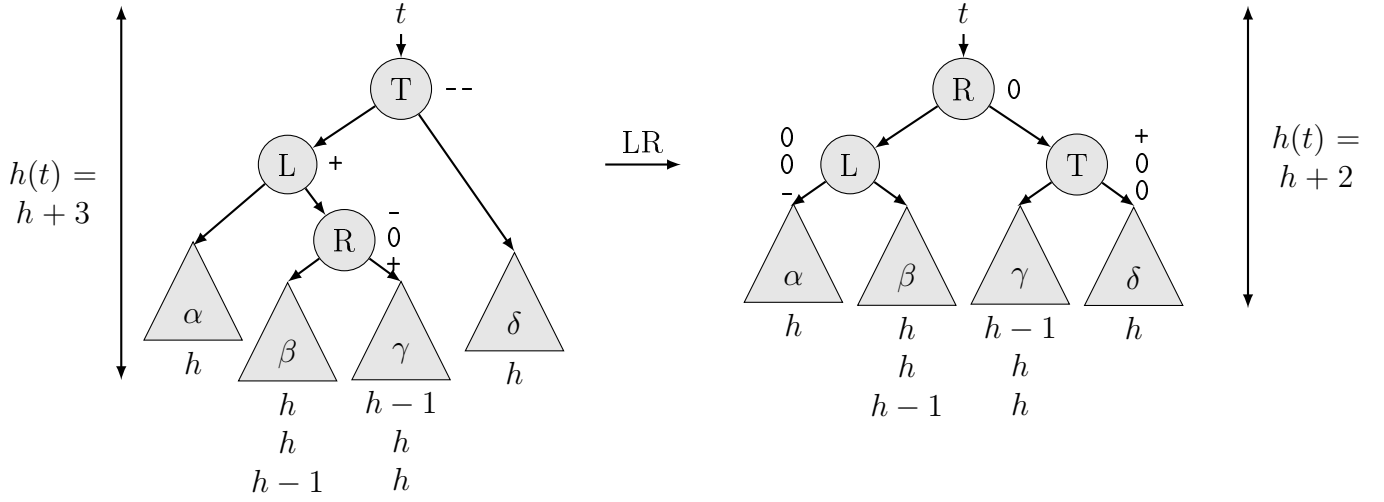


Figure 5: rotation $(--, +)$.

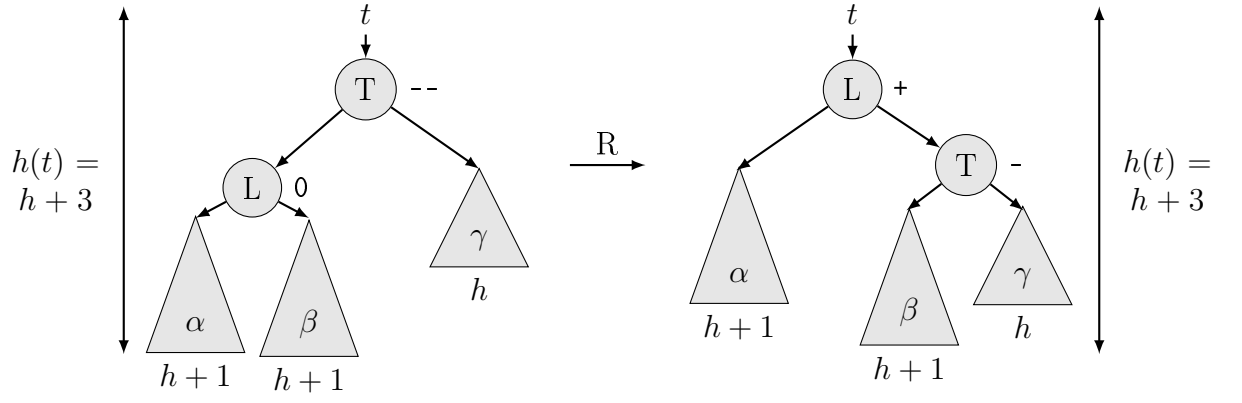


Figure 6: rotation $(--, 0)$.

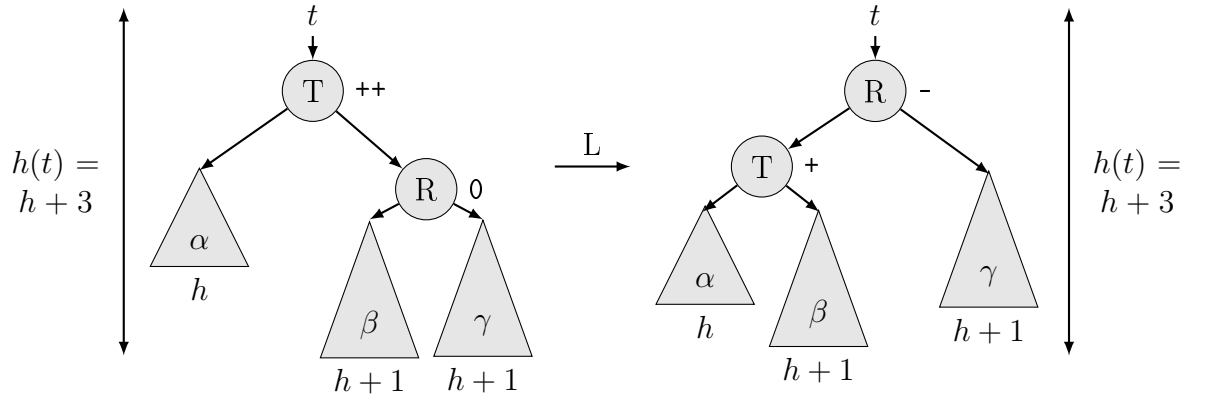


Figure 7: rotation $(++, 0)$.

$3 > 2$, so a new leaf ③ is inserted into the right, empty subtree of node ②.
 Now we are at the new leaf ③. The actual subtree was empty. After inserting key 3, The actual subtree is (3). It has become higher.
 Considering the balance information explicitly stored, this is the whole tree:
 $\{ [() 2 \circ (3)] 4 + [(6) 8 \circ (10)] \}$. It is not rebalanced yet.
 (Some of) the ancestors of the new leaf must be rebalanced.
 We go up in the tree. First we arrive at node ②.
 Its right subtree became higher, so its balance is increased by one.
 Now, this is the whole tree: $\{ [() 2 + (3)] 4 + [(6) 8 \circ (10)] \}$.
 The actual subtree is $[() 2 + (3)]$, and it has become higher.
 We step up in the tree. We arrive at the root node: ④.
 Its left subtree has become higher, so its balance is reduced by one.
 The actual subtree is the whole tree: $\{ [() 2 + (3)] 4 \circ [(6) 8 \circ (10)] \}$.
 We have finished rebalancing, and we can see that the result of insertion is a balanced BST, i.e. an AVL tree. Thus, we have also finished insertion. We do not need any rotation here.

Exercise 2.14 Insert key 1 into AVL tree $\{ [2] 4 + [(6) 8 \circ (10)] \}$, and show the details.

Example 2.15 Insert key 9 into AVL tree $\{ [2] 4 + [(6) 8 \circ (10)] \}$.

Solution: We insert key 9. The key of the root is 4.
 $9 > 4$, so we go to the right subtree: $[(6) 8 \circ (10)]$.
 $9 > 8$, so we go to the right subtree: (10).
 $9 < 10$, so we go to its left subtree, which is empty.
 Thus we put the new node here: $\{ [2] 4 + [(6) 8 \circ (\langle 9 \rangle 10 \circ \langle \rangle)] \}$.
 Then, we rebalance the ancestors at each tree level but stop when the first imbalanced node is found.
 The subtree corresponding to this imbalanced node is the whole tree now:
 $\{ [2] 4 ++ [(6) 8 + (\langle 9 \rangle 10 - \langle \rangle)] \}$.
 We can use rotation $(++, +)$ now (see also Figure 2):

$$[\alpha T ++ (\beta R + \gamma)] \rightarrow [(\alpha T \circ \beta) R \circ \gamma]$$

Notice that in these rotation schemes, the Greek letters are the subtrees, and the English uppercase letters followed by the balance signs (like $++, +, \circ$, etc.) are the keys of the nodes.

We use the rotation scheme above on the imbalanced BST above where
 $\alpha = [2] = (2)$; $T = 4$; $\beta = (6)$; $R = 8$; $\gamma = (\langle 9 \rangle 10 - \langle \rangle) = [(9) 10 - ()]$.
 Finally, we receive the AVL tree $\{ [(2) 4 \circ (6)] 8 \circ [(9) 10 - ()] \}$.

Example 2.16 Let us see an example of rotation $(++, -)$. Insert key 7 into AVL tree $\{ [2] 4+ [(6) 8\circ (10)] \}$.

Solution: To keep the BST property, we find the appropriate empty subtree and insert node $\textcircled{7}$ there.

We receive tree $\{ [2] 4+ [(\langle \rangle 6\circ \langle 7 \rangle) 8\circ (10)] \}$. Then we go up and rebalance the ancestors of the new leaf $\textcircled{7}$ at each tree level, but we stop when the first imbalanced node is found. We receive the following imbalanced BST:

$\{ [2] 4++ [(\langle \rangle 6+ \langle 7 \rangle) 8- (10)] \}$.

Rotation $(++, -)$ is needed (see also Figure 4):

$$\{ \alpha T++ [(\beta L-\circ+ \gamma) R- \delta] \} \rightarrow \{ [\alpha T\circ\circ- \beta] L\circ [\gamma R+\circ\circ \delta] \}$$

In this $(++, -)$ rotation scheme, $T++$ is the root of the imbalanced BST. Its right child is $R-$. The left child of R is L .

L is the right-left grandchild of the imbalanced node $T++$. After the rotation, L becomes the new root of this (sub)tree. L 's parent and grandparent nodes become its two children. and the four subtrees: α , β , γ and δ remain in their original order. The inorder traversal of the whole (sub)tree remains the same.

L may have the balance $-$, $\circ$, or $+$. After rotation, the new balance of T and R depends on the old balance of L . If the old balance of L was $-$, the new balance of T will be $\circ$, and that of R will be $+$, etc. (See Figure 4 for more details.) We use this later rotation scheme on the previous imbalanced BST $\{ [2] 4++ [(\langle \rangle 6+ \langle 7 \rangle) 8- (10)] \}$ where $\alpha=[2]=(2)$; $T=4$; $\beta=\circ$; $L=6$; $\gamma=\langle 7 \rangle=(7)$; $R=8$; $\delta=(10)$. The old balance of $L=6$ was $+$, so the new balance of $T=4$ will be $-$, and the new balance of $R=8$ will be $\circ$. Applying this rotation rule, we receive AVL tree $\{ [(2) 4- ()] 6\circ [(7) 8\circ (10)] \}$.

Let us notice, if the balance of node L was b before applying this $(++, -)$ rotation rule, then (after applying this rotation rule) the new balance of T will be b_t , and that of R will be b_r where

$$b_t = -\lfloor (b+1)/2 \rfloor \quad \text{and} \quad b_r = \lfloor (1-b)/2 \rfloor.$$

We get similarly the rotation rules of the cases when the balance of the root of the smallest imbalanced subtree is $--$ (see also Figures 3 and 5.):

$$\begin{aligned} & [(\alpha L- \beta) T-- \gamma] \rightarrow [\alpha L\circ (\beta T\circ \gamma)] \\ & \{ [\alpha L+ (\beta R-\circ+ \gamma)] T-- \delta \} \rightarrow [(\alpha L\circ\circ- \beta) R\circ (\gamma T+\circ\circ \delta)] \\ & b_t = -\lfloor (b+1)/2 \rfloor \quad \text{and} \quad b_r = \lfloor (1-b)/2 \rfloor \end{aligned}$$

where b was the balance of node R before applying the $(--, +)$ rotation rule; but after applying this rule, the new balance of L will be b_l , and that of T will be b_t .

In the $(--, +)$ rotation scheme, $T--$ is the root of the imbalanced BST. Its left child is $L+$. The right child of L is R :

R is the left-right grandchild of the imbalanced node $T--$. After the rotation, R becomes the new root of this (sub)tree. R 's parent and grandparent nodes become its two children. and the four subtrees: α , β , γ and δ remain in their original order. The inorder traversal of the whole (sub)tree remains the same.

Let us see the detailed codes of insertion. Compared to the insert procedure of BSTs, we use an extra parameter here: d is a reference parameter of Boolean type. After the call, it is true if the height of the actual subtree increased by one, and it is false if its height did not change.

AVLinsert(&t:Node* ; k:ℤ ; &d:ℬ)					
t = ∅					
t := new Node(k) d := true	k < t → key		k > t → key		ELSE
	AVLinsert(t → left, k, d)		AVLinsert(t → right, k, d)		d := false
	d		d		
	leftSubTreeGrown (t, d)	SKIP	rightSubTreeGrown (t, d)	SKIP	

leftSubTreeGrown(&t:Node* ; &d:ℬ)		
t → b = −1		
l := t → left		t → b := t → b − 1
l → b = −1		
rotateMMm(t, l)	rotateMMp(t, l)	d := (t → b < 0)
d := false		

rightSubTreeGrown(&t:Node* ; &d:ℤ)		
$t \rightarrow b = 1$		
$r := t \rightarrow right$		$t \rightarrow b := t \rightarrow b + 1$
$r \rightarrow b = 1$		
$rotatePPp(t, r)$	$rotatePPm(t, r)$	$d := (t \rightarrow b > 0)$
$d := false$		

rotatePPp(&t, r : Node*)

$t \rightarrow right := r \rightarrow left$
$r \rightarrow left := t$
$r \rightarrow b := t \rightarrow b := 0$
$t := r$

rotateMMm(&t, l : Node*)

$t \rightarrow left := l \rightarrow right$
$l \rightarrow right := t$
$l \rightarrow b := t \rightarrow b := 0$
$t := l$

rotatePPm(&t, r : Node*)

$l := r \rightarrow left$
$t \rightarrow right := l \rightarrow left$
$r \rightarrow left := l \rightarrow right$
$l \rightarrow left := t$
$l \rightarrow right := r$
$t \rightarrow b := -\lfloor (l \rightarrow b + 1)/2 \rfloor$
$r \rightarrow b := \lfloor (1 - l \rightarrow b)/2 \rfloor$
$l \rightarrow b := 0$
$t := l$

rotateMMp(&t, l : Node*)

$r := l \rightarrow right$
$l \rightarrow right := r \rightarrow left$
$t \rightarrow left := r \rightarrow right$
$r \rightarrow left := l$
$r \rightarrow right := t$
$l \rightarrow b := -\lfloor (r \rightarrow b + 1)/2 \rfloor$
$t \rightarrow b := \lfloor (1 - r \rightarrow b)/2 \rfloor$
$r \rightarrow b := 0$
$t := r$

Notice that the rotation rules $(--, 0)$ and $(++, 0)$ on Figures 6 and 7 are never applied in insertion. (They will be applied in deletion.)

One insertion applies maximum one of the rotation rules on Figures 2-5 and whole once because a successful insertion creates a new leaf, and then we go up level by level in the tree rebalancing the ancestors of the fresh leaf until

- we find an imbalanced node. In this case, the height of the actual subtree has been increased by one. Then we apply the appropriate rule, i.e. one of those on Figures 2-5. And each of them decreases the height of the actual subtree by one. (See Figures 2-5 for the details.) Thus, the

original height of this subtree has been restored, and we leave insertion. Consequently, **in insertion, we never modify the balances of the nodes outside of the smallest imbalanced subtree.**

- rebalancing a node, we find that it is perfectly balanced. This means that one of its direct subtrees has grown to the height of the other. Therefore, the insertion has not changed the height of the subtree corresponding to the actual node. Thus, we have finished rebalancing. We did not find an imbalanced node, and the insertion was completed without rotation.
- we arrive at the root of the whole tree but do not find an imbalanced node. This case is similar to the previous one: no rotation is needed because the BST remains balanced.

Example 2.17 *Let us see an example of the first case above, especially when the imbalanced node is not the root of the whole tree. Insert key 7 into AVL tree $\{ [(1) 2- ()] 3+ [(\langle 4 \rangle 5\circ \langle 6 \rangle) 8- (9)] \}$.*

Solution: After inserting key 7, but still before rebalancing, we have the following tree: $\{ [(1) 2- ()] 3+ [(\langle 4 \rangle 5\circ \langle \{ \} 6\circ \{7\})) 8- (9)] \}$.

We rebalance nodes ⑥, ⑤, and ⑧. We stop rebalancing here, because node ⑧-- has become imbalanced (thus node ③+ is not rebalanced):

$\{ [(1) 2- ()] 3+ [(\langle 4 \rangle 5+ \langle \{ \} 6+ \{7\})) 8-- (9)] \}$

Next we apply rotation (--, +) at node ⑧--.

The actual rule of rotation is

$[(\alpha L+ \langle \beta R+ \gamma \rangle) T-- \delta] \rightarrow [(\alpha L- \beta) R\circ (\gamma T\circ \delta)]$

with $\alpha=\langle 4 \rangle$, $L=5$, $\beta=\circ$, $R=6$, $\gamma=\{7\}=\langle 7 \rangle$, $T=8$, $\delta=(9)=\langle 9 \rangle$

(see also Figure 5).

The result: $\{ [(1) 2- ()] 3+ [(\langle 4 \rangle 5- \langle \rangle) 6\circ (\langle 7 \rangle 8\circ \langle 9 \rangle)] \}$.

Exercise 2.18 *Let us modify the Node type of AVL trees so that we store the height of the corresponding subtree in each node, but we do not keep the balances.*

The four more straightforward rules of rotations on figures 2, 3, 6, 7 can be reduced to two rules + recalculating the heights of the modified subtrees. The two simple rotations:

Right-to-left rotation: $[\alpha T (\beta R \gamma)] \rightarrow [(\alpha T \beta) R \gamma]$

Left-to-right rotation: $[(\alpha L \beta) T \gamma] \rightarrow [\alpha L (\beta T \gamma)]$

Which one should be used in the different cases?

The more complex rules of rotations, which correspond to figures 4 and 5, can be considered double rotations now because we can get them as two

simple rotations applied at the appropriate points of the tree + recalculating the heights of the modified subtrees.

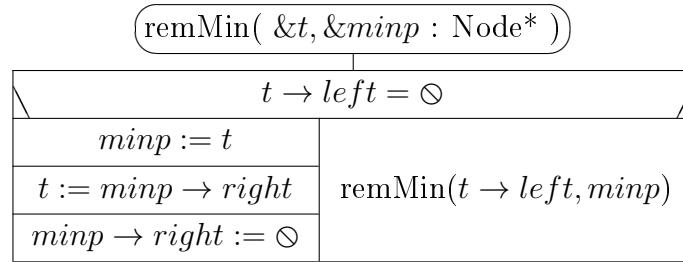
How do we use the simple rotations above in the different cases?

Exercise 2.19 At the structogram (and C/C++) level, implement insertion using the new rules of rotations developed in Exercise 2.18.

Compare your implementation of insertion to the corresponding structograms given above. What can we say about the complexity of the code? What can we tell about efficiency issues?

2.2 AVL trees: removing the minimal (maximal) node

On nonempty BSTs, procedure remMin removes the minimal node (i.e. the node with minimal key):



Applying this to AVL tree $\{ [2] 4+ [(6) 8 \circ (10)] \}$, we receive the imbalanced BST $\{ [] 4++ [(6) 8 \circ (10)] \}$ while minp refers to the minimal node of the original tree where $\text{minp} \rightarrow \text{key} = 2 \wedge \text{minp} \rightarrow \text{left} = \text{minp} \rightarrow \text{right} = \ominus$. Notice that we show the theoretical balances above. At the representation level, the code above does nothing with the balances (b attributes). After removing the minimal node, we have to go up from the place of it and, level by level, rebalance the ancestors. The appropriate rotation must be applied if we find a node with a “++” balance. (Notice that balance “--” is impossible when we remove the minimal node.)

Rotation $(++, 0)$ is needed here (see Figure 7 for the details):

$$\{ \alpha \text{ T}++[\beta \text{ R} \circ \gamma] \} \rightarrow \{ [\alpha \text{ T}+ \beta] \text{ R}- \gamma \}.$$

Notice that this rotation does not modify the height of the actual subtree. Therefore, no more rebalancing is needed after this rotation, even if this rotation is applied to a proper subtree of the whole BST. The final result is $\{ [() 4+ (6)] 8- (10) \}$ here.

Notwithstanding, when balancing $(++, +)$ or $(++, -)$ is needed, the rotated subtree becomes lower, and after the rotation, rebalancing continues.

Then, it is possible that two or more balancing are required while going up in the tree. However, each balancing needs constant time, and the number of levels is less than $1.45 \log n$. Thus, the maximal runtime is still $\Theta(\log n)$.

Example 2.20 *Remove the minimal node from the AVL tree*

$\{ [(1) 2+ (\langle \rangle 3+ \langle 4 \rangle)] 5+ [(\langle 6 \rangle 7+ \langle \{8\} 9- \{ \}) 10- (\langle \rangle 11+ \langle 12 \rangle)] \}$.

We need two rotations here.

Solution: We remove the leftmost node, i.e. node ①. The appropriate subtree becomes empty. Its height is decreased by one. Then, the balance of node ② becomes “++”. The subtree rooted by node ② needs rotation $(++,+)$: $[() 2++ (3+ \langle 4 \rangle)] \rightarrow [(2) 3 \circ (4)]$. The height of this subtree is decreased by one. (Rotation $(++,+)$ always decreases the height of the rotated subtree by one.) Consequently, the balance of node ⑤ is increased by one. The whole tree now:

$\{ [(2) 3 \circ (4)] 5++ [(\langle 6 \rangle 7+ \langle \{8\} 9- \{ \}) 10- (\langle \rangle 11+ \langle 12 \rangle)] \}$.

The right child of node ⑤++ is ⑩−. Thus, we need rotation $(++, -)$.

The left child of ⑩− is ⑦+. After rotation $(++, -)$ (see Figure 4 for the details), the right-left grandchild of the root of the rotated subtree becomes its root. This is ⑦+ now. Its original parent and grandparent become its two children, and the four subtrees are arranged in order:

$\{ [(\langle 2 \rangle 3 \circ \langle 4 \rangle) 5- (6)] 7 \circ [(\langle 8 \rangle 9- \langle \rangle) 10 \circ (\langle \rangle 11+ \langle 12 \rangle)] \}$.

The balances of the three nodes are determined by the old balance of the original grandson (⑦+ here), according to Figure 4. The four subtrees are not changed. (Just the brackets were rewritten here for better reading.)

Based on the rules discussed above, we can complete the code of procedure `remMin( $t, minp$ )` above. For AVL trees, we need an extra Boolean parameter d , which is *true*, **iff** the height of the appropriate subtree has been decreased by one. (It cannot be reduced by two or more.) When we return from a recursive call, we check d . If it is *true*, we have to go on with rebalancing and possibly rotations. If d is *false*, neither subsequent rebalancing nor rotations are needed.

AVLremMin(&t, &minp:Node* ; &d:ℤ)		
$t \rightarrow left = \ominus$		
$minp := t$	AVLremMin($t \rightarrow left, minp, d$)	
$t := minp \rightarrow right$		
$minp \rightarrow right := \ominus$	d	
$d := true$	leftSubTreeShrunk(t, d)	SKIP

leftSubTreeShrunk(&t:Node* ; &d:ℤ)	
$t \rightarrow b = 1$	
rotate_PP(t, d)	$t \rightarrow b := t \rightarrow b + 1$
	$d := (t \rightarrow b = 0)$

rotate_PP(&t:Node* ; &d:ℤ)		
$r := t \rightarrow right$		
$r \rightarrow b = -1$	$r \rightarrow b = 0$	$r \rightarrow b = 1$
rotatePPm(t, r)	rotatePP0(t, r)	rotatePPp(t, r)
	$d := false$	

rotatePP0(&t, r : Node*)
$t \rightarrow right := r \rightarrow left$
$r \rightarrow left := t$
$t \rightarrow b := 1$
$r \rightarrow b := -1$
$t := r$

Exercise 2.21 Based on procedure AVLremMin($t, minp, d$) above, write the structograms of procedure AVLremMax($t, maxp, d$). You will need the code of rotation rules $(--, -)$, $(--, +)$ and $(--, 0)$ defined on Figures 3, 5 and 6. You should write new code for rotation rule $(--, 0)$ defined in Figure 6.

2.3 AVL trees: deletion

Procedures $\text{del}(t, k)$ and $\text{delRoot}(t)$ for BSTs:

$\text{del}(\&t:\text{Node}^* ; k:\mathcal{T})$			
$t \neq \ominus$			
$k < t \rightarrow \text{key}$	$k > t \rightarrow \text{key}$	$k = t \rightarrow \text{key}$	SKIP
$\text{del}(t \rightarrow \text{left}, k)$	$\text{del}(t \rightarrow \text{right}, k)$	$\text{delRoot}(t)$	

$\text{delRoot}(\&t:\text{Node}^*)$		
$t \rightarrow \text{left} = \ominus$	$t \rightarrow \text{right} = \ominus$	$t \rightarrow \text{left} \neq \ominus \wedge t \rightarrow \text{right} \neq \ominus$
$p := t$	$p := t$	$\text{remMin}(t \rightarrow \text{right}, p)$
$t := p \rightarrow \text{right}$	$t := p \rightarrow \text{left}$	$p \rightarrow \text{left} := t \rightarrow \text{left}$
		$p \rightarrow \text{right} := t \rightarrow \text{right}$
delete p	delete p	delete $t ; t := p$

These are completed below as it was done in the case of procedure $\text{AVLremMin}(t, \text{minp}, d)$ in section 2.2. We add Boolean parameter d , which is *true*, **iff** the height of the appropriate subtree has been decreased by one. (It cannot be reduced by two or more.) When we return from a recursive call, we check d . If it is *true*, we have to go on with rebalancing and possibly rotations. If d is *false*, neither subsequent rebalancing nor rotations are needed.

AVLdel(&t:Node* ; k:ℳ ; &d:ℬ)					
t ≠ ⊙					
k < t → key		k > t → key		k = t → key	
AVLdel(t → left, k, d)		AVLdel(t → right, k, d)		AVLdelRoot(t, d)	
d		d			
leftSubTreeShrunk(t, d)	SKIP	rightSubTreeShrunk(t, d)	SKIP		
d := false					

AVLdelRoot(&t:Node* ; &d:ℤ)			
$t \rightarrow left = \ominus$	$t \rightarrow right = \ominus$	$t \rightarrow left \neq \ominus \wedge t \rightarrow right \neq \ominus$	
$p := t$	$p := t$	rightSubTreeMinToRoot(t, d)	
$t := p \rightarrow right$	$t := p \rightarrow left$		
delete p	delete p	d	
$d := true$	$d := true$	rightSubTreeShrunk(t, d)	SKIP

rightSubTreeMinToRoot(&t:Node* ; &d:ℤ)
AVLremMin($t \rightarrow right, p, d$)
$p \rightarrow left := t \rightarrow left ; p \rightarrow right := t \rightarrow right ; p \rightarrow b := t \rightarrow b$
delete $t ; t := p$

When rotation $(++, 0)$ or $(--, 0)$ is performed, it does not modify the height of the actual subtree. Therefore, no more rebalancing or rotations are needed after one of these rotations, even if the rotation is applied to a proper subtree of the whole BST.

Notwithstanding, when rotation $(++, +)$, $(--, -)$, $(++, -)$ or $(--, +)$ is needed (see Figures 2-5), then the rotated subtree becomes lower, and after the rotation, rebalancing goes on. Then, two or more rotations may be needed while going up in the tree. However, each rotation needs constant time, and the number of levels is less or equal to $1.45 \log n$. Thus, the maximal runtime of deletion is also $\Theta(\log n)$.

Exercise 2.22 Based on procedure leftSubTreeShrunk(t, d), write procedure rightSubTreeShrunk(t, d) together with its auxiliary procedures. (See rotation rule $(--, 0)$ given on Figure 6.)

Example 2.23 Delete key 2 from AVL tree

$\{ [(1) 2+ (\langle \rangle 3+ \langle 4 \rangle)] 5+ [(\langle 6 \rangle 7+ \langle \{8\} 9- \{\})] 10- (\langle \rangle 11+ \langle 12 \rangle)] \}$.

Solution: We delete node ②. AVLremMin is called on its right subtree, and node ③ is removed from it to be AVL tree (4). No rotation is needed during AVLremMin, but the height of this right subtree is decreased by one, so AVLremMin returns with $d = true$. Node ② is substituted by node ③, which inherits the balance of node ②. Then its balance is decreased by one

because $d = \text{true}$. Thus the actual subtree becomes $[(1) 3 = (4)]$.

The height of this subtree has been decreased by one. Consequently, the balance of node ⑤ is increased by one. The whole tree now:

$\{ [(1) 3 \circ (4)] 5++ [(\langle 6 \rangle 7+ \langle \{8\} 9- \{ \})) 10- (\langle \rangle 11+ \langle 12 \rangle)] \}$.

The right child of node ⑤ $++$ is ⑩ $-$. Thus, we need rotation $(++, -)$.

The left child of ⑩ $-$ is ⑦ $+$. After rotation $(++, -)$ (see Figure 4 for the details), the right-left grandchild of the root of the rotated subtree becomes its root. This is ⑦ $+$ now. Its original parent and grandparent become its two children, and the four subtrees are arranged in order:

$\{ [(\langle 1 \rangle 3 \circ \langle 4 \rangle) 5- (6)] 7 \circ [(\langle 8 \rangle 9- \langle \rangle) 10 \circ (\langle \rangle 11+ \langle 12 \rangle)] \}$.

The balances of the three nodes are determined by the old balance of the original grandchild (⑦ $+$ here), according to Figure 4. The four subtrees are not changed. (Just the brackets were rewritten here for better reading.)

Example 2.24 *Delete key 5 from AVL tree*

$\{ [(1) 2+ (\langle \rangle 3+ \langle 4 \rangle)] 5+ [(\langle 6 \rangle 7+ \langle \{8\} 9- \{ \})) 10- (\langle \rangle 11+ \langle 12 \rangle)] \}$.

Solution: To delete node ⑤, first AVLremMin is called on its right subtree: $[(\langle 6 \rangle 7+ \langle \{8\} 9- \{ \})) 10- (\langle \rangle 11+ \langle 12 \rangle)]$. AVLremMin is called recursively on its left subtree: $(\langle 6 \rangle 7+ \langle \{8\} 9- \{ \})$. We remove its leftmost node ⑥. The balance of its parent ⑦ $+$ is increased by one. The corresponding subtree is $(7++ \langle \{8\} 9- \{ \})$. Rotation $(++, -)$ is performed on it. The result is $(\langle 7 \rangle 8 \circ \langle 9 \rangle)$. Its height is decreased by one, so the recursive call to AVLremMin returns with $d = \text{true}$, and the balance of ⑩ $-$ is increased by one. The corresponding subtree is $[(\langle 7 \rangle 8 \circ \langle 9 \rangle) 10 \circ (\langle \rangle 11+ \langle 12 \rangle)]$.

Its height is decreased by one, so the first call to AVLremMin returns with $d = \text{true}$. After the run of AVLremMin, the whole tree is

$\{ [(1) 2+ (\langle \rangle 3+ \langle 4 \rangle)] 5+ [(\langle 7 \rangle 8 \circ \langle 9 \rangle) 10 \circ (11+ \langle 12 \rangle)] \}$.

Node ⑤ is substituted by node ⑥:

$\{ [(1) 2+ (\langle \rangle 3+ \langle 4 \rangle)] 6+ [(\langle 7 \rangle 8 \circ \langle 9 \rangle) 10 \circ (\langle \rangle 11+ \langle 12 \rangle)]$.

$d = \text{true}$ still shows that the height of its right subtree was decreased by one, so the balance of ⑥ $+$ is also decreased by one. The final AVL tree is

$\{ [(1) 2+ (\langle \rangle 3+ \langle 4 \rangle)] 6 \circ [(\langle 7 \rangle 8 \circ \langle 9 \rangle) 10 \circ (\langle \rangle 11+ \langle 12 \rangle)]$.

Exercise 2.25 *Write an alternative procedure code AVLdel(t, k, d). It shall be similar to the previous one, but if key k is in a node with two children, it shall substitute this node with the maximal node of its left subtree.*

3 General trees

Compared to binary trees, a node of a *rooted tree* may have an unlimited (although finite) number of children. A rooted tree is *ordered* if the order of the children of nodes is important. An ordered rooted tree is also called *general tree*. (See Figure 8.) We will not define empty subtrees of *general trees*.

Using *general trees*, we can model hierarchical structures like

- directory hierarchies in computers
- (block) structure of programs
- different kinds of expression in mathematics and computer science
- family trees, etc.

General trees can be represented with linked binary trees in a natural way. The type `Node` of the nodes of general trees is given below.

- Pointer *firstChild* refers to the first child of a node.
 $p \rightarrow \text{firstChild} = \emptyset$, **iff** $\text{node}(*p)$ is a leaf.
- Pointer *nextSibling* refers to the next sibling in a list of children.
 $p \rightarrow \text{nextSibling} = \emptyset$, **iff** $(*p)$ is the tree's root node or the last sibling in a list of children.

Node
+ <i>firstChild, nextSibling</i> : Node*
+ <i>key</i> : $\mathcal{T}$
+ Node() { <i>firstChild</i> := <i>nextSibling</i> := $\emptyset$ }
+ Node($x:\mathcal{T}$) { <i>firstChild</i> := <i>nextSibling</i> := $\emptyset$; <i>key</i> := x }

In the nodes, there might be *parent* pointers referring to the parent of the children. We will not use parent pointers here.

Exercise 3.1 Try to invent alternative (linked) representations of general trees. Compare your representations to the linked binary model above. Consider memory needs and flexibility.

In the textual or parenthesised representation of general trees, we start with the root of the actual (sub)tree. Thus, a nonempty tree fits the general scheme

$$(R \ t_1 \dots t_n)$$

where R is the content of the root node and $t_1 \dots t_n$ are the direct subtrees of node R .

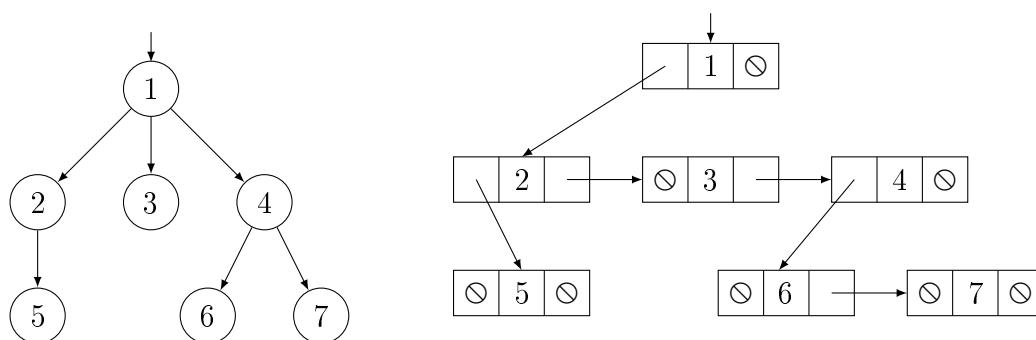


Figure 8: The abstract structure of a general tree on the left and its linked binary representation on the right. Its textual or parenthesized representation is $\{ 1 [2 (5)] (3) [4 (6) (7)] \}$. (The different kinds of brackets may vary.)

Example 3.2 In general tree $\{ 1 [2 (5)] (3) [4 (6) (7)] \}$, 1 is in the root. Its children are the nodes with keys 2, 3 and 4. The corresponding subtrees are $[2 (5)]$, (3) and $[4 (6) (7)]$ in order. The leaves of the tree are the nodes with keys 5, 3, 6 and 7. (See Figure 8.)

Exercise 3.3 Write a procedure printing a general tree given in linked binary representation. The result should be in a textual representation.

Write another procedure to build a general tree in linked binary representation. Its input is a text file with the tree in parenthesised form.

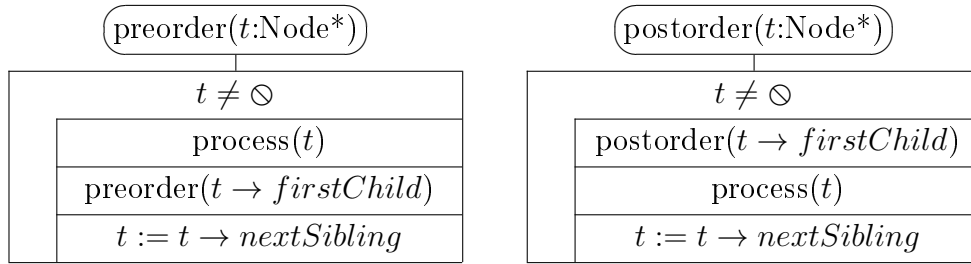
Write these printing and reading procedures for the linked representations you invented in Exercise 3.1.

(The printing procedures are usually more straightforward than those which build up the linked representation from the textual form.)

To make the *preorder* traversal of a general tree, we have to traverse its binary representation also with *preorder* traversal where *firstChild* corresponds to *left* and *nextSibling* corresponds to *right*.

But the *postorder* traversal of a general tree requires the *inorder* traversal of its binary representation.⁶

The *preorder* and *postorder* traversals of a general tree are given below, provided that it is given in linked binary representation.



Exercise 3.4 Write the traversals of general trees above for the representations you invented while solving Exercise 3.1.

Given a general tree in some representation, write a procedure copying it into another given representation.

4 B+ trees and their basic operations

See file **B+trees.pdf** in Canvas.

⁶A search in the file system of a computer requires *preorder* traversal while evaluating an expression requires *postorder* traversal of the general (i.e. theoretical) expression tree.